ScalpTrack â€” Detailed Project Specification

Version: 1.0
Prepared: 13 October 2026
Team: Three students
Delivery format: Jupyter notebooks first; a basic frontend only after the notebook pipeline works
Planning horizon: One week or less
Document status: Implementation plan. No datasets have been downloaded, models trained, or project notebooks executed as part of writing this specification.

1. Agreed direction and unresolved assumptions

1.1 Decisions confirmed by the team

Decision	Agreed scope
Project domain	Hair and scalp computer vision
Main requested result	Scalp-condition classification
Additional requested results	Hair count, hair thickness, and 3D measurements
Available real data	Only the public dataset sources linked in the discussion
New photography or equipment	No collection of new scalp photographs or stereo captures is assumed
Time available	One week or less
Academic requirement	Demonstrate every technique in Experiments 1â€“9 and develop an integrated CV application for Experiment 10
First implementation	Python/OpenCV and .ipynb notebooks
Frontend	Deferred; add a small wrapper only if time and presentation needs justify it

1.2 Assumptions used to make the plan actionable

- CPU execution is the baseline. GPU or Colab access was not confirmed.
- The exact submission date is unknown. The seven-day schedule is illustrative and must be compressed if less time remains.
- No dedicated skin-disease dataset or skin deliverable has been selected. General skin analysis is a later phase; scalp skin is already part of the current image analysis.
- Manual annotation of a new mask/count dataset was not confirmed. It is not a mandatory dependency for the first submission.
- Synthetic images may be generated locally as labeled algorithm-verification fixtures. They are not real scalp data, do not enter the clinical-condition training/test dataset, and cannot establish performance on human scalp images.
- No additional real-image dataset beyond the linked sources is silently added.
- The phrase â€œsuch asâ€ in Experiment 10 is interpreted as examples of applications. The integrated scalp analyzer is the selected application. Webcam face detection, panorama stitching, and gesture recognition are not separate mandatory projects under this interpretation. If the faculty requires every example application, that is an additional scope requirement, not something this document marks as completed.

1.3 Feasibility boundary

The selected public listings support a classification/detection project. They do **not establish**, from the information inspected, the availability of individually labeled hairs, physical pixel scale, or calibrated scalp stereo pairs. Their actual downloadable exports still require inspection.

Therefore the first submission has two connected parts:

1. **A working scalp-condition classifier and explainable image-processing pipeline.**
2. **Experimental hair-measurement and stereo modules with explicit validity/status reporting.**

The measurement modules must be implemented and demonstrated where suitable inputs exist. Their biological interpretation and units must match the data. The plan does not promise true hairs/cmÂ², hair diameter in micrometres, or real scalp depth in millimetres from arbitrary single photographs.

If all three physical measurements are compulsory on real scalp images, the current data restriction is a blocker. More suitable labeled/calibrated data would be required; additional model complexity does not remove that requirement.

2. Project identity, motivation, and problem statement

Working name: ScalpTrack
Recommended subtitle: Explainable Scalp Condition Classification and Experimental Hair Measurement

Problem statement:

> Develop a reproducible notebook-based computer vision system that analyzes public hair/scalp images, predicts dataset-defined scalp-condition labels, and visualizes the image-processing evidence behind the analysis. Apply the complete CV lab syllabus through preprocessing, filtering, edge detection, color conversion, threshold segmentation, texture analysis, image pyramids, stereo disparity, and Haar face detection. Investigate visible hair counting and apparent shaft width, while reporting physical and 3D measurements only when their required data and calibration are available.

The motivation is accessible visual analysis and a strong academic demonstration. A reduction in treatment cost, faster clinical diagnosis, or treatment effectiveness is not an evaluated outcome of this one-week project.

The attached ScalpTrack image provides the product inspiration: scalp analysis, explainable texture/segmentation, and eventually progress tracking. Its proposed â€œhealth index,â€ pose standardization by Haar, and 3D hair-part volume require additional evidence; they are not accepted as established capabilities.

2.1 Intended users and workflow

The first user is a student, evaluator, or researcher inspecting public dataset images.

1. Select an approved dataset image in a notebook.
2. Inspect the image and select an appropriate region of interest (ROI).
3. Run preprocessing and compare the lab techniques.
4. View the predicted condition and all class scores.
5. Inspect segmentation, edges, and texture features.
6. Run hair measurements if the ROI passes the necessary checks.
7. View a stereo result only when a valid pair is supplied, or switch to the clearly labeled synthetic stereo demonstration.
8. Export figures and a structured result record.

A medical diagnosis, prescription, serum recommendation, or automated treatment plan is outside this workflow.

3. Deliverable scope and priorities

Priority	Work	Meaning of completion
P0	Data audit and reproducible manifests	Real downloaded labels, counts, source versions, and splits are documented
P0	All named algorithms in Experiments 1â€“9	Each algorithm runs, produces figures, and has a short explanation and limitations
P0	Scalp-condition classification baseline	At least one trained classifier with a held-out evaluation appropriate to the data
P0	Hair count and apparent width prototype	Defined quantities, quality checks, synthetic verification, and honest results/status on selected real ROIs
P0	Stereo disparity/depth subsystem	Validates inputs and computes disparity; synthetic calibrated example when real scalp stereo is absent
P0	Integrated notebook application	Acquisition/loading â†’ processing â†’ features â†’ prediction/measurements â†’ visualization/export
P1	Lightweight transfer-learning classifier	MobileNetV2 comparison if compute, downloads, and data quality permit
P1	Error analysis and one controlled ablation	Explain failures and measure the contribution of a chosen feature/preprocessing change
P1	Visible hair/scalp coverage and flake-candidate coverage	Exploratory segmentation-derived descriptors where suitable images exist
P2	Grad-CAM, VGG19 comparison, extra interactive controls	Add after P0 is stable; these are not prerequisites for the application
Later	Frontend, longitudinal tracking, general skin analysis	Separate extensions after this submission

P0 requires a functioning implementation and appropriate evidence. It does not require an unsupported numerical result for every public photo. An unavailable measurement with a precise reason is valid behavior; it is not evidence that the corresponding real-world measurement has been achieved.

4. Dataset investigation and selection

4.1 Sources already supplied in the discussion

Public metadata was checked on 13 October 2026. A listing's image count is not a verified count of unique patients or independent original images.

Source	What was inspectable	Proposed use	What remains to verify
[Kaggle Hair Diseases â€” VGG19 reference notebook](https://www.kaggle.com/code/sundarannamalai/hair-diseases-vgg19/)	Notebook title/page; cell contents were not available through the reader	Reference approach, not an evaluated implementation	Actual input dataset, labels, split method, preprocessing, license, and availability of notebook code
[Kaggle Hair Diseases â€” CNN reference notebook](https://www.kaggle.com/code/emammame/hair-diseases-cnn/notebook)	Notebook title/page; cell contents were not available through the reader	Second reference approach	Same checks as above; do not assume it uses the same input dataset
[Kaggle Hair Diseases dataset](https://www.kaggle.com/datasets/sundarannamalai/hair-diseases)	Dataset page/title located; file inventory and labels were not inspectable	First candidate for image-level classification	Confirm that this is actually the linked VGG19 notebook's input; inspect class folders, originals, counts, and license
[Litmers scalp](https://universe.roboflow.com/litmers-cwywf/scalp-bo6xy)	Indexed public listing: object detection, 37 images, 10 classes, CC BY 4.0	Small demonstration set for scalp appearance, white-flake candidates, and failure cases	Exact chosen export/version, image resolution, boxes, any available masks/count/scale metadata
[Maheshwariss HAIR AND SCALP](https://universe.roboflow.com/maheshwariss-workspace-nn7d1/hair-and-scalp-w3ih8)	Public listing: object detection, 4,800 images, four classes, CC BY 4.0	Candidate fallback for annotated-region condition classification	Unique originals, augmented derivatives, label quality, export access, and original grouping
[Aditi hair-problem](https://universe.roboflow.com/aditi-xltkw/hair-problem-xt4pi/browse?queryText=&pageSize=50&startingIndex=0)	Contents could not be verified	Unselected candidate pending inspection	Task type, classes, number of originals, license, and annotations
[Shared Claude discussion](https://claude.ai/share/30e934d4-a7b0-4511-b9e3-3f87f3c38b49) and [LinkedIn short link](https://lnkd.in/p/dSHPbDm8)	No usable project content was verified	Context references only	Do not infer a dataset, implementation, or result from inaccessible links

The Maheshwariss listing names four labels: **Contact Dermatitis**, **Hair-and-Scalp-Diseases-MPB-**, **Lichen-Planus**, and **Psoriasis**. Preserve these raw names in provenance. Do not silently reinterpret the unusual MPB label.

The indexed Litmers listing names: **Clear_scalp**, **Dry_surface**, **Healthy Scalp**, **Oily**, **Pimples**, **Redness**, **Shiny_surface**, **Skin_trouble**, **White_flakes**, and **Yellow_sebum**. These mix visible signs and condition-like labels. They are not a ten-class mutually exclusive disease taxonomy.

4.2 Day 1 selection rule

Choose **one primary source** for the condition classifier.

1. Inspect the Kaggle reference notebook's actual input and its dataset export.
2. Use it for image-level classification only if usable image labels, license/access, and an independent split are available.
3. Otherwise inspect the Maheshwariss export. Use labeled bounding-box crops for **region-level classification**, with all crops from an original image kept in one split.
4. Do not collapse multiple differently labeled boxes into a single image class.
5. Treat Litmers as supplementary demonstrations; 37 listed images across ten labels do not support a convincing independent ten-class disease classifier.
6. Keep Aditi unselected until its data is verified.

If only detection labels are usable, the notebook and report must say **annotated-region classification**. Performance on ground-truth box crops does not establish automatic whole-photo diagnosis or automatic disease localization.

4.3 Data manifest

Create one row per original image, with derived-crop rows linked to it where necessary.

Required fields:

- `image_id`, `original_id`, `relative_path`, `dataset_source`, `dataset_version`
- `license`, `download_date`, `raw_label`, `canonical_label`, `task_type`
- `width_px`, `height_px`, `channels`, `file_hash`, `duplicate_group_id`
- `patient_id` if actually supplied; otherwise null
- `split`: train, validation, test, demo, or excluded
- `roi_id`, `parent_image_id`, `bbox_xyxy` for derived crops
- `has_hair_instance_labels`, `has_count_labels`, `has_segmentation_masks`
- `pixel_scale_um_per_px`, `stereo_pair_id`, `calibration_id` if documented; otherwise null
- `exclusion_reason` and `notes`

Do not fill absent patient identity, physical scale, or stereo relationships with guesses.

4.4 Split and leakage policy

- Prefer supplied patient/group identities. Otherwise group by original image and known derivatives.
- Check exact hashes and candidate perceptual duplicates. Review suspicious clusters; a perceptual hash alone is not proof that all related images have been found.
- Keep resized images, augmented copies, repeated web images, and all crops from one original in the same split.
- Target approximately 70/15/15 train/validation/test **by independent group**, with class representation where feasible.
- Preserve a publisher's split only after checking for cross-split derivatives and duplicates.
- Generate new augmentations after the split and only for training.
- Fit scalers, imputation, feature selection, model parameters, and any learned preprocessing on training data.
- Select hyperparameters, confidence rules, and preprocessing using validation data.
- Use test data once for the final comparison. Demonstration images are identified separately; their selection must not drive model tuning.
- Record the achieved split counts. Do not force the target fractions when the groups are too few.
- If independent test support is inadequate, report exploratory group cross-validation rather than claiming a robust held-out test.
- Unknown patient identities remain a limitation even when original-image grouping is correct.

Do not merge all linked datasets merely to increase the image count. Their label meanings, capture settings, and duplicates may differ.

5. Requirements and lab traceability

5.1 Core requirements

ID	Requirement
REQ-01	Load approved images, validate them, and retain original pixel geometry
REQ-02	Demonstrate every named image-processing method in Experiments 1â€“7
REQ-03	Extract documented numerical features and compare their distributions
REQ-04	Train and evaluate a condition classifier with an appropriate split
REQ-05	Define and implement an exploratory visible-hair counting method
REQ-06	Estimate apparent shaft width in pixels where image quality supports it
REQ-07	Enable physical width/density only with documented scale and suitable annotations/geometry
REQ-08	Compute stereo disparity and calibrated depth on a valid stereo input
REQ-09	Implement Haar face detection with a no-face/manual-ROI path
REQ-10	Present integrated results, overlays, feature tables, and exportable records
REQ-11	Mark missing inputs, ambiguous masks, and unsupported measurements explicitly
REQ-12	Provide a reproducible environment, saved model/configuration, and faculty-facing explanations

5.2 Every experiment mapped to this project

Lab	CO	Required techniques	Concrete scalp-project role	Notebook and evidence
1	CO1	Reading, displaying, resizing, grayscale, pixel analysis	Audit resolution; compare

scalp/hair patch intensities; preserve geometry for measurement | `00_data_audit_basics.ipynb`: image grid, dimensions, pixel table, histograms, resize comparison |
2	CO1	Mean filter, Gaussian blur, Laplacian sharpening	Compare denoising and detail enhancement before edges/thresholding	`01_filters_color.ipynb`: original/filtered panels and documented parameter comparison
3	CO1	Sobel, Prewitt, Canny	Expose shaft/flake boundaries and measure edge sensitivity	`02_edges_segmentation.ipynb`: gradient maps, Canny maps, edge-density/continuity observations
4	CO1	RGB â†’ grayscale, HSV, LAB	Separate intensity/color cues; compare representations for features and candidate masks	`01_filters_color.ipynb`: individual channels and patch statistics
5	CO2	Global and adaptive thresholding	Create exploratory hair/scalp/flake-candidate masks for measurements	`02_edges_segmentation.ipynb`: fixed, Otsu, adaptive mean, adaptive Gaussian, and overlay comparisons
6	CO2	Statistical texture contrast, energy, entropy	Quantify appearance and build an interpretable classifier baseline	`03_texture_pyramids.ipynb`: GLCM features, histogram entropy, distributions, feature CSV
7	CO2	Gaussian and Laplacian pyramids	Show scale-dependent hair/flake detail and verify reconstruction	`03_texture_pyramids.ipynb`: levels, residuals, reconstruction, error table
8	CO3	Stereo disparity and depth estimation	Implement a geometry module; demonstrate it on a calibrated synthetic pair if real scalp pairs are absent	`06_stereo_depth.ipynb`: pair, disparity, validity mask, depth, point cloud, ground-truth comparison; real-scalp capability status
9	CO4	Haar Cascade face detection	Detect visible faces in linked full-head images; suggest a rough crop and support manual ROI selection	`07_face_roi.ipynb`: boxes or explicit no-face result, ROI overlay, applicability limits
10	CO1â€“CO4	Acquisition, processing, feature extraction, result visualization in a CV application	Integrate the project into one notebook-based scalp analyzer	`08_integrated_demo.ipynb`: complete image-to-result workflow and exported figures/JSON/CSV

Hair measurement and classification add depth to the syllabus coverage. Stereo and Haar remain distinct modules with valid inputs; they are not falsely presented as prerequisites for every close-up scalp image.

6. Architecture and data flow

The analysis has separate branches because a classifier, a pixel-width measurement, and a stereo reconstruction have different input requirements.

```mermaid
flowchart TD
    A["Approved image and metadata"] --> B["Validate quality and select ROI"]
    B --> C["Lab processing and feature extraction"]
    C --> D["Condition classifier"]
    C --> E["Candidate hair masks"]
    E --> F["Count and pixel width checks"]
    S["Stereo pair and calibration"] --> T["Stereo geometry checks"]
    T --> U["Disparity and valid depth"]
    D --> R["Results, figures, and status"]
    F --> R
    U --> R
```

6.1 Shared conventions

- OpenCV loading produces BGR. Convert explicitly to RGB for display and any RGB-based model preprocessing.
- Keep an immutable original image after EXIF orientation correction.
- Represent ROIs as `(x0, y0, x1, y1)` in original-image pixels, with the maximum coordinates exclusive.
- Classifier inputs may be resized; thickness and counting use the native-resolution ROI.
- Record all crop/resize transforms. Isotropic resize scale must be inverted before reporting an original-pixel measurement.
- Keep analysis arrays separate from display-normalized images.
- Convert to floating point before signed differences, Laplacians, pyramid residuals, or multiplication that could overflow uint8.
- Use one documented boundary policy for filtering comparisons.
- Store seed, configuration, package versions, model identifier, and input hash with each run.

6.2 Minimal shared functions

The following are planned interfaces, not existing code:

Function	Purpose
`load_image(path)`	Validate/decode and return RGB image plus metadata
`validate_roi(image_shape, roi)`	Bound-check a native-pixel ROI
`assess_quality(rgb_roi, config)`	Report blur/exposure/resolution flags
`compare_filters(gray_roi, config)`	Mean, Gaussian, and Laplacian-sharpened outputs
`compare_edges(gray_roi, config)`	Sobel, Prewitt, Canny, and comparison summaries
`segment_candidates(rgb_roi, config)`	Candidate masks plus method/quality metadata
`extract_features(rgb_roi, config)`	Fixed-schema color, edge, histogram, and texture features
`estimate_visible_hairs(rgb_roi, hair_mask, config)`	Count/crossing estimate with its precise interpretation
`estimate_shaft_width(hair_mask, config)`	Native-pixel width statistics or an unavailable result
`predict_condition(rgb_roi, model_bundle)`	Ordered class scores and model decision
`detect_faces(rgb_image, cascade, config)`	Face boxes and applicability status
`compute_stereo(left, right, calibration, config)`	Disparity, validity, depth/point cloud where supported
`analyze_image(path, roi, model_bundle, config)`	Assemble the single-image analysis record
`export_result(result, output_dir)`	JSON, flattened CSV, and labeled figure files

Stereo is called separately or through an optional pair argument. A single-image request never invents a partner image.

7. Notebook contracts

Each notebook must have: purpose, relevant lab numbers, input assumptions, methods/parameters, visible outputs, numerical checks where appropriate, failure cases, conclusions, and references. It must run from a fresh kernel using shared configuration.

Notebook	Main work	Required saved artifacts
`00_data_audit_basics.ipynb`	Source audit, label mapping, independent groups/splits, reading/display/resizing/grayscale/pixels	Dataset manifest, split manifest, class/count plots, example grid, capability audit
`01_filters_color.ipynb`	Mean/Gaussian/Laplacian comparison; grayscale/HSV/LAB analysis	Filter/channel contact sheets, parameter table, patch-statistics CSV
`02_edges_segmentation.ipynb`	Sobel/Prewitt/Canny; global/adaptive masks and conservative cleanup	Edge panels, mask overlays, threshold sweep, failure-case panels
`03_texture_pyramids.ipynb`	GLCM/histogram texture, distributions, Gaussian/Laplacian levels	Feature CSV, distribution plots, pyramid figures, reconstruction error
`04_hair_measurements.ipynb`	Count/crossing and apparent shaft-width prototypes	Synthetic truth comparisons, count-line overlays, width overlays/histograms, real-image status table
`05_classification.ipynb`	Classical baseline; optional MobileNetV2; independent evaluation and ablation	Saved model bundle, class map, run config, metrics JSON, confusion matrix, failure examples
`06_stereo_depth.ipynb`	Pair validation, matching, disparity, calibrated depth, synthetic verification	Synthetic provenance/calibration, disparity/depth figures, point-cloud view, error/coverage table
`07_face_roi.ipynb`	Haar detection on linked images; manual crop fallback	Face/ROI overlays, no-face cases, positive-test availability status
`08_integrated_demo.ipynb`	One clean examiner-facing application	Analysis JSON/CSV, compact result panel, technique comparison gallery, optional widget controls

The final demo loads saved models. It must not retrain a network during a presentation.

8. Image-processing implementation details

8.1 Basic operations and quality assessment

Show file dimensions, datatype, channel ordering, dynamic range, and a small pixel neighborhood. Compare nearest-neighbor, bilinear, and area resizing on the same ROI; these comparisons explain why display/classifier resizing must be separated from measurement.

Quality descriptors:

- Variance of the grayscale Laplacian as an empirical blur indicator.
- Dark and near-saturated pixel fractions.
- Native ROI size and apparent shaft resolution.
- Specular/highlight and severe shadow observations.
- ROI relevance and whether a face/full-head view or scalp close-up is being analyzed.

Do not adopt a universal blur cutoff without validation. Initial thresholds are configuration choices, explored on development images. Quality flags may disable measurement without automatically disabling classification.

8.2 Spatial filters

Use the same original grayscale ROI for all comparisons.

- Mean filtering: initial kernel sizes 3Ã—3 and 5Ã—5.
- Gaussian blur: initial 3Ã—3/5Ã—5 kernels and documented sigma.
- Laplacian sharpening: signed Laplacian and `I_sharp = I - alpha * Laplacian(I)`, using OpenCV's negative-centre second-derivative convention and a small documented alpha.
- Clip only the final display/export image; preserve signed/intermediate float arrays.

Compare detail retention, noise amplification, edge response, and effect on downstream candidate masks. In a synthetic noise experiment, report MSE/PSNR against the known clean fixture. For real images without a clean reference, do not claim PSNR improvement.

Aggressive sharpening is not the default input to shaft-width measurement. It can change the mask boundary.

8.3 Color conversion

Display RGB, grayscale, H/S/V, and L/a/b channels individually, with suitable scales.

For OpenCV uint8 conversions, document HSV hue encoding and the offset/scaling of LAB channels. Float LAB has different numerical conventions. Do not compare raw uint8 a-channel values with textbook signed a* without conversion.

Compare selected patches using mean, standard deviation, quantiles, and histograms. LAB/HSV descriptors can be classifier features or candidate-segmentation aids; a red pixel alone is not proof of inflammation, and a bright patch alone is not proof of oil or flakes.

8.4 Edges

- Sobel: signed x/y derivatives, magnitude, optional orientation.
- Prewitt: explicit horizontal/vertical 3Ã—3 kernels through `cv2.filter2D`; show the kernels and magnitude.
- Canny: Gaussian preprocessing followed by several documented low/high threshold pairs.

Normalize only visual copies. Compare edge pixel fraction, fragmentation, response to background texture, and continuity on the same ROI. Sobel/Prewitt strengths require a documented threshold before a binary-map comparison with Canny.

Edge detection locates image transitions. It does not identify follicles or provide a biological hair count by itself.

8.5 Threshold segmentation

Mandatory comparisons:

1. Fixed global threshold.
2. Otsu global threshold.
3. Adaptive mean threshold.
4. Adaptive Gaussian threshold.

Adaptive block sizes must be odd and larger than one; initial sweep: 15, 31, and 51 where the ROI size permits. Record the constant C and whether normal or inverse thresholding is used.

Candidate categories:

- **Hair:** dark, thin structures in a suitable scalp close-up.
- **Visible scalp:** candidate background within the manually selected relevant ROI.

- **Flakes:** bright local candidates with additional size/shape/color checks.
- **Unknown/excluded:** shadows, highlights, lesions, background, or ambiguous regions.

Use optional opening/closing and connected-component filtering conservatively. Display the mask before and after cleanup. Kernels must not erase narrow shafts or artificially thicken them.

The masks are heuristic candidates unless matched to ground-truth masks. Global darkness fails on shadows and light/gray hair; brightness fails on glare and pale skin. Do not report IoU/Dice against another threshold method as if it were ground truth.

8.6 Coverage descriptors

These are supporting descriptors, not the team's primary promised outputs.

For disjoint hair/scalp candidate masks H and S:

- Visible scalp fraction: `|S| / |H union S|`.
- Hair coverage fraction: `|H| / |H union S|`.
- Scalp-to-hair area ratio: `|S| / |H|`.
- Flake-candidate coverage: candidate flake pixels divided by a named valid ROI area.

Return unavailable when the denominator is empty. Display excluded/unknown area separately. Pixel-area fraction is not hairs/cmÂ², and flake-candidate area is not a clinical percentage of dandruff or disease severity.

9. Texture and pyramids

9.1 Texture feature definition

Use grayscale values quantized to 32 levels initially. Document resizing/patch size, quantization, and whether features describe a whole ROI or a selected scalp/hair patch.

Compute GLCMs at distances 1, 2, and 4 pixels and angles 0Â°, 45Â°, 90Â°, and 135Â°. Normalize each matrix and report a fixed aggregation, such as mean and standard deviation across distances/angles.

For normalized GLCM P:

- Contrast: `sum((i-j)^2 * P[i,j])`.
- ASM: `sum(P[i,j]^2)`.
- Energy: `sqrt(ASM)`.
- GLCM entropy in bits: `-sum(P[i,j] * log2(P[i,j]))` over positive entries.
- Optional homogeneity and correlation.

Also calculate histogram entropy separately and label it explicitly. Do not mix histogram entropy and GLCM entropy in one field.

If masks are used, count only valid pixel pairs, or use valid interior patches. Filling everything outside a mask with zero creates artificial texture. Standard whole-array GLCM calls do not automatically exclude those filled pixels.

Compare distributions across dataset labels and selected example patches. Contrast, energy, and entropy are statistical descriptors, not validated scales of scalp hydration, cleanliness, or health. Test whether they improve the classifier instead of assigning biological interpretations by assumption.

9.2 Gaussian and Laplacian pyramids

Build a Gaussian pyramid using repeated downsampling and a Laplacian pyramid using signed differences between each Gaussian level and the expanded next level.

- Start with level 0 and up to four reductions, stopping before the ROI becomes too small.
- Use explicit destination sizes when expanding odd-dimension images.
- Preserve float residuals; visualize their signs with a suitable display scale.
- Reconstruct from the coarsest level by expansion and addition of residuals.
- Report reconstruction maximum absolute error and MSE in the original intensity units.

Explain how thin hair/flake detail disappears at coarser levels. Pyramids support multiscale analysis; they do not create additional real samples or recover lost microscopic detail.

10. Hair-count specification

10.1 Quantities must have distinct names

Quantity	Definition	Requirements	First-submission status
Isolated visible-strand candidate count	Number of separate, shaft-like components in a defined ROI after documented filtering	Clear close-up, well-separated shafts, reliable candidate mask	Exploratory where suitable images exist
Reference-line crossing count	Number of separate shaft-mask intersections with a specified line	Good contrast, visible shafts, stable line placement	Exploratory fallback, reported as crossings
Follicle count	Number of actual follicular openings in the ROI	Resolvable openings and suitable labeled evidence	Not established by current source listings
Hair count per physical area	Defined hairs/root instances divided by calibrated area	Valid count definition, instance/root labels, scale and geometry	Unavailable unless the export supplies these prerequisites

A connected component is not automatically one hair. A branched skeleton is not automatically several hairs. A follicle and a visible shaft are different counting units. One curling shaft can cross the reference line more than once.

10.2 Baseline algorithm

1. Select a native-resolution scalp close-up ROI.
2. Inspect candidate masks from the threshold comparisons.
3. Choose a documented mask method using development images, not the final test set.
4. Skeletonize the candidate hair mask.
5. Record connected components, endpoint/junction counts, length, and simple elongation measures.
6. Use component counting only when shafts are separated and branching/fragmentation is limited.
7. For a crossing count, place a documented line perpendicular to the dominant visible shaft direction; count distinct intersection runs after a small, specified merge rule.
8. Display the counting line, masks, numbered candidates, and excluded/ambiguous regions.
9. Return a quantity name, value, ROI/line coordinates, quality flags, and method parameters.

Do not silently relabel crossings as a count of unique biological hairs. Do not count Canny's two boundaries of a shaft as two hairs.

10.3 Verification

Generate deterministic synthetic masks/images containing known numbers of separated straight and curved shafts. Include deliberately difficult fixtures: touching shafts, crossings, fragmented masks, noise, and varying contrast.

- On simple separated fixtures, the candidate-component count should match the known component count.
- On simple reference-line fixtures, the crossing count should match known intersections.
- Challenging fixtures must show errors/ambiguity rather than being quietly omitted.
- On real public images without count labels, save overlays and describe the count as unvalidated/exploratory.
- Real count MAE is reported only if appropriate ground-truth counts actually exist in the export.

The module contributes an explainable estimate and a failure analysis. It does not certify true follicle density.

11. Hair-thickness specification

11.1 First supported measurement

The first supported output is **apparent 2D shaft width in original-image pixels**.

â€œThickness in micrometresâ€ remains unavailable unless a documented spatial scale applies to that image/ROI. Image dimensions, camera megapixels, apparent face width, or an arbitrary average hair diameter do not supply that scale.

11.2 Baseline algorithm

1. Work on the native-resolution ROI.

2. Obtain the conservative hair candidate mask.
3. Compute the Euclidean distance transform inside that mask.
4. Sample it along the skeleton, using approximately `width_px = 2 * distance_to_background`.
5. Exclude tips, junction neighborhoods, overlapping/touching components, and clearly unreliable boundaries.
6. Report median, interquartile range, valid sample count, and distribution/overlay.
7. Flag narrow shafts whose projected width is below the configured resolution requirement. Start by investigating a minimum of about four pixels; treat this as an empirical development choice.
8. Compare width stability across a small development-only threshold sweep.

The distance-transform formula is an approximate discrete-image width estimator. Rasterization and segmentation introduce bias; verify and document it rather than treating each sample as an exact diameter.

An optional check measures edge-to-edge distance perpendicular to the local skeleton tangent on well-isolated segments. Add it only if the baseline is stable.

11.3 Units and validity

With a documented local scale `s_um_per_px`:

`width_um = width_px * s_um_per_px`.

Record where the scale came from and whether it applies at the same image plane and resolution. Account for cropping/resizing; a nonuniform resize invalidates a single scalar conversion unless its geometry is explicitly handled.

Validate on synthetic shafts with known pixel widths, multiple orientations, and blur/noise levels. Report pixel-width MAE and orientation-dependent bias. A reasonable initial development target is an error within roughly 1â€“2 pixels on clearly resolved simple fixtures; record actual results and failures.

For real images without validated masks or diameter labels, apparent widths remain exploratory. Projected image width is not necessarily the true cylindrical diameter of a hair.

12. Stereo and 3D specification

12.1 Real-data gate

Before attempting a real scalp result, verify:

- Two views correspond to the same static scalp region.
- Their relative camera geometry is documented or can legitimately be established.
- Intrinsics, distortion parameters, extrinsics/baseline, and units are available for metric depth.
- Rectification/correspondence is valid and sufficient texture is present.
- Resolution, occlusion, glare, and repeated hair patterns permit useful matching.

A single photo is not stereo. Two unrelated disease images are not stereo. A shifted copy of a real photo does not create evidence of real scalp depth.

If the linked exports contain no valid pair, record **real scalp stereo: unavailable â€” no verified stereo pair/calibration**. The code and synthetic experiment still proceed.

12.2 Default one-week demonstration

Generate a deterministic synthetic 3D scene with two or three textured surfaces at known depths. Render both cameras from the **same scene** using a known pinhole model and baseline.

Save:

- Scene geometry, seed, texture generation settings, and units.
- Left/right camera parameters and images.
- Ground-truth depth/disparity and occlusion/visibility masks.
- A clear â€œsynthetic algorithm demonstrationâ€ label on all results.

A curved scalp-like surface is optional after the simple scene works. It is a visualization of synthetic geometry, not a reconstruction of a person's scalp.

Synthetic fixtures add no new human-image source. They demonstrate the computation required by Lab 8. **Faculty acceptance of synthetic-only stereo coverage remains unconfirmed.** If real stereo images are compulsory, this lab's real-data requirement remains blocked under the present restriction.

12.3 Matching and depth pipeline

1. Validate shape, pair metadata, and calibration; convert to grayscale.
2. Rectify real inputs with documented calibration. The synthetic cameras may already be rectified.
3. Run `cv2.StereoSGBM_create`; optional StereoBM comparison if time allows.
4. Use a positive multiple of 16 for `numDisparities` and an odd block size. Record disparity search range and all matching parameters.
5. Convert OpenCV's fixed-point output to float disparity pixels by dividing by 16.
6. Construct validity masks for invalid disparity, borders, occlusion, and unreliable matches; include left-right consistency when feasible.
7. Use the calibrated Q matrix with `reprojectImageTo3D`, or the equivalent rectified-camera equations.
8. Mask invalid depth/points; save actual numerical arrays separately from color-normalized displays.

For an ideal rectified pair with equal principal-point x coordinates:

`Z = f_px * B / d_px`.

Z has the baseline's units. The general calibrated case uses Q and accounts for the principal-point offset. Do not compute metric Z from arbitrary guessed focal length or baseline.

12.4 Outputs and evaluation

Required figures:

- Left and right images.
- Disparity in pixels with a color bar.
- Validity/coverage mask.
- Depth with its units, or an explicit unavailable-depth result.
- A sparse point-cloud view with labeled axes, when calibration supports it.
- Ground-truth versus estimated disparity/depth for the synthetic scene.

Required numerical summaries:

- Valid coverage over a defined ground-truth-visible evaluation region.
- Disparity MAE and percentage exceeding a declared error threshold.
- Depth MAE/RMSE in the known synthetic units over valid pixels.
- Invalid/occluded pixel fractions.

Do not improve reported error simply by hiding most difficult pixels; display error and coverage together. A point cloud describes reconstructed visible surfaces, not hidden follicle depth, true hair volume, or 3D shaft diameter.

13. Haar face detection and ROI support

Use the OpenCV frontal-face Haar Cascade, loaded from the installed cascade assets. Verify the classifier loaded successfully.

- Convert an eligible linked image to grayscale.
- Optionally equalize its histogram for this face-detection branch only.
- Run `detectMultiScale` with documented scale factor, neighbors, and minimum size.
- Draw detected boxes and return their coordinates.
- Offer a bounded rough upper-head crop proposal on suitable frontal photographs.
- Require manual ROI confirmation for scalp analysis.
- If no face is present/detected, continue through the manual ROI path.

Haar does not standardize head pose, detect individual hairs, or reliably find the scalp in trichoscopic close-ups. A proposed crop above a face is a heuristic, not scalp segmentation.

Search for an eligible face-containing example **within the approved linked data**. Include a positive example only if one actually exists and is detected. If none is available, demonstrate the no-face path and mark positive detection verification as data-limited. Do not import unrelated

face datasets or pretend a blank result demonstrates positive face-detection accuracy.

This is Lab 9 coverage through implementation and applicability analysis. A faculty requirement for a demonstrated positive detection would need an eligible image; it remains a stated input dependency if the linked sources lack one.

14. Scalp-condition classification

14.1 Task definition

Select the final label set from the audited primary dataset, not from assumptions about what the Kaggle notebooks contain.

- For one label per image: supervised multiclass image classification.
- For one label per derived box crop: supervised region classification.
- For multiple independent labels on an image: use an explicit multilabel task, or use well-defined region crops. Do not pretend those labels are mutually exclusive.
- A healthy/normal class is included only if the chosen dataset actually provides it.
- Low-confidence/uncertain is a decision state, not a fabricated ground-truth class.

Record label provenance, any exclusions, label-to-index order, and the prediction input mode.

14.2 Mandatory classical baseline

Extract a compact fixed feature vector:

- Grayscale summary statistics and histogram entropy.
- Selected HSV/LAB channel statistics or small fixed histograms.
- GLCM contrast, energy, entropy, and optionally homogeneity/correlation.
- Edge density and simple orientation summaries.
- Optional candidate-area fractions only if the segmentation definition is stable.

Start with a `StandardScaler + LogisticRegression` pipeline, a fixed seed, balanced class weighting where useful, and a small validation-selected regularization search. A random forest is a fallback if scaling/linearity is unsuitable; avoid a large model search.

Also evaluate a training-derived majority-class baseline. If there are too few independent groups for the selected features/classes, reduce model complexity and report exploratory performance.

All preprocessing and feature-schema decisions are serialized with the model. Test-time feature order must match training.

14.3 Transfer learning, if feasible

Use one lightweight model first: **MobileNetV2 with ImageNet-pretrained weights**.

- Input: RGB, documented 224Ã—224 geometry, and MobileNetV2-specific preprocessing.
- Start with a frozen backbone and a small classification head, or cache embeddings and train a simple classifier.
- Preserve the task's image/region mode across training and inference.
- Apply modest augmentation to training only, after split grouping.
- Benchmark a small extraction batch before committing to all images.
- Cap training work to a practical portion of one day; checkpoint progress.
- Optional limited fine-tuning uses a smaller learning rate and careful batch-normalization handling.
- Record whether the selected result uses a frozen backbone or fine-tuning.

GPU availability is an optimization, not a prerequisite for the P0 classifier. If framework/weight downloads or extraction are infeasible, retain the working classical baseline and record why the transfer-learning experiment was omitted.

VGG19 and a custom CNN are references/stretch comparisons. Building several networks from scratch, a large ensemble, or a new segmentation model is not part of the one-week baseline.

14.4 Prediction presentation

Show the predicted dataset label, model/task name, and an ordered class-score table/bar chart.

Raw probability-like scores are model scores; clinical correctness probabilities have not been

established. If an uncertainty threshold is used, choose it on validation data and document what it does.

A manually selected ROI classifier must not be described as an automatic whole-head detector. Optional Grad-CAM is a visualization of model sensitivity, not a lesion mask, flake percentage, or proof that the model reasoned correctly.

15. Evaluation and experiment design

15.1 Classification report

For the final independent evaluation, report:

- Number of independent original groups and derived inputs per split/class.
- Accuracy, macro F1, balanced accuracy, and weighted F1.
- Per-class precision, recall, F1, and support.
- Confusion matrix, majority baseline, and selected model settings.
- Inference timing with hardware, input size, and measured batch conditions.
- Representative correct predictions and failure cases.

Report unsupported classes and missing test support plainly. Very high accuracy on augmented copies or duplicated web images is not convincing evidence of generalization.

A fixed accuracy promise such as 95% is not an acceptance criterion. The goal is reproducible evidence and an honest comparison. If a model fails to beat the baseline, explain the result and data limitations.

15.2 One required controlled comparison

Use the same independent groups and split to compare:

- Classical features **with** texture descriptors.
- The same classifier **without** texture descriptors.

Select settings on validation data and lock them before final evaluation. This gives Lab 6 a measurable role beyond displaying a feature table.

If time permits, compare classical features with frozen MobileNetV2 features on that same task and split. Do not compare metrics from unrelated publishers' notebooks as if they were matched experiments.

15.3 Algorithm versus real-image validation

Module	Evidence available without new annotation	Evidence required for a real-world accuracy claim
Filter denoising	Known clean/noisy synthetic fixture; real-image visual comparison	Paired clean reference or a justified external evaluation protocol
Edges/segmentation	Method comparison, synthetic masks, real overlays	Appropriate edge/pixel-mask ground truth
Hair count	Known synthetic components/crossings; real exploratory overlays	Real count/instance labels with the same counting definition
Shaft width	Known synthetic pixel widths; real apparent-width overlays	Validated masks/diameter measurements and scale for physical units
Stereo	Known synthetic calibration/depth; real pair processing if available	Real pair, trustworthy calibration, and depth reference for accuracy
Haar	Runtime output on eligible linked images; no-face behavior	Face-containing positive inputs/labels for detection evaluation
Condition classifier	Audited published labels and independent split	Clinical adjudication and external data for clinical-performance claims

Synthetic verification checks algorithm behavior. It does not replace real-world validation.

15.4 Failure cases to show

Include at least one applicable example of:

- Dark shadows mistaken for hair.
- Bright glare mistaken for flakes.

- Light/gray or low-contrast hair missed by dark-threshold segmentation.
- Crossing/touching shafts causing count/width ambiguity.
- Blur reducing apparent shaft resolution.
- A scalp close-up containing no face.
- Missing stereo/scale metadata causing unavailable metric output.
- Misclassification, class imbalance, or a dataset-label ambiguity.

16. Result schema and unavailable behavior

Every numerical measurement includes a value, units, status, method, and explanation. Suggested statuses:

- `ok`: computation succeeded under its stated prerequisites; this is not a clinical-validity label.
- `exploratory`: useful provisional image result without adequate real ground truth.
- `synthetic_demo`: generated verification input.
- `unavailable`: necessary data/calibration/quality is absent.
- `failed`: runtime or validation failure.
- `not_requested`: branch was not run.

Unknown/unavailable values are **null**, not zero. Zero has a different meaning: a valid measurement found none.

The following is a schema illustration, not an analyzed image:

```json
{
  "schema_version": "1.0",
  "image_id": "example_from_manifest",
  "source": "primary_dataset",
  "roi_xyxy": [0, 0, 100, 100],
  "input_hash": null,
  "run_config_id": "saved_config",
  "quality": {
    "flags": [],
    "measurement_eligible": null
  },
  "classification": {
    "task_mode": "image_or_region_as_audited",
    "predicted_label": null,
    "class_scores": {},
    "model_id": null,
    "status": "not_requested"
  },
  "hair_count": {
    "quantity": "reference_line_crossings",
    "value": null,
    "unit": "crossings",
    "status": "not_requested",
    "method": null,
    "reason": "Schema example only"
  },
  "shaft_width": {
    "median_px": null,
    "iqr_px": null,
    "valid_samples": 0,
    "median_um": null,
    "pixel_scale_um_per_px": null,
    "status": "unavailable",
    "method": null,
    "reason": "No documented scale or validated analysis in this example"
  },
  "stereo": {
    "pair_id": null,
    "calibration_id": null,
    "disparity_unit": "px",
    "depth_unit": null,
    "valid_coverage": null,
    "status": "unavailable",
```

```
    "reason": "No verified stereo pair/calibration"
  },
  "artifacts": {}
}
```

Runtime results add the texture feature vector, face boxes, segmentation method, coverage descriptors, counting-line coordinates, saved overlays, and timings as appropriate.

For the actual result, width in pixels and width in physical units have separate validity flags. A missing physical scale must not erase an otherwise usable exploratory pixel-width result.

17. Integrated demo and visual presentation

The integrated notebook is the main presentation surface.

17.1 Single-image result panel

Show:

1. Original image and native ROI.
2. Condition prediction with a class-score chart.
3. Candidate mask and edge overlay.
4. Count/crossing visualization with a precise quantity label.
5. Shaft-width summary with pixel units and validity status.
6. A small texture-feature table.
7. Available/unavailable capability badges with short reasons.

Additional tabs/cells show filter/color comparisons and pyramid reconstruction.

17.2 Stereo result panel

Keep the stereo demonstration visibly separate from a single public photo's analysis. Show pair provenance, calibration source, input images, disparity, valid depth, and point cloud.

Synthetic captions must appear on the figure itself and in saved filenames/metadata. A screenshot of synthetic geometry must not look like a measured result from the selected public scalp photo.

17.3 Interaction

Start with a notebook cell for selecting a manifest image and ROI coordinates. Add simple `ipywidgets` controls only after static execution is reliable.

Useful controls: image selector, ROI bounds, threshold method, adaptive block size/C, Canny thresholds, and crossing-line position. Interactive changes are demonstrations; they do not rewrite the locked evaluation configuration.

Export Matplotlib/OpenCV figures as PNG, numerical features as CSV, and the full result as JSON. A compact comparison gallery and a reproducible report provide the â€œextraâ€ polish expected from a three-person group.

18. Planned repository and environment

18.1 File organization

This table is a proposed implementation layout. Only this specification is delivered at the planning stage.

Path	Purpose
`detailed_spec.md`	This agreed implementation specification
`README.md`	Setup, data access, execution order, scope, and demo instructions
`requirements.txt`	Tested base environment pinned after initial compatibility verification
`requirements-training.txt`	Optional TensorFlow training profile
`configs/default.yaml`	Parameters, seeds, thresholds, ROI/feature settings, output paths
`notebooks/`	The nine notebooks listed in Section 7
`src/scalptrack/`	Shared loading, processing, features, measurement, classification, stereo, face, and export functions
`data/raw/`	Immutable downloaded linked-source exports

`data/manifests/`	Source/version, label, group, split, and capability records
`data/derived/`	Crops and cached features with parent provenance
`data/synthetic/`	Verification fixtures with known truth, stored separately from real data
`models/`	Saved baseline/optional transfer models and class/feature metadata
`outputs/figures/`	Labeled presentation figures
`outputs/results/`	Per-image JSON/CSV, metrics, experiment summaries
`outputs/demo/`	Frozen demonstration results and presentation-ready gallery

18.2 Environment

Use Python 3.11 or another mutually compatible installed version. Verify package compatibility on Day 1 and pin actual tested versions rather than speculative version combinations.

Base packages: NumPy, OpenCV (headless for notebook/Colab use), SciPy, scikit-image, scikit-learn, pandas, Matplotlib, Pillow, joblib, PyYAML, Jupyter, nbformat/nbclient, and optional ipywidgets.

Use TensorFlow/Keras only for the optional MobileNetV2 path. Use one neural-network framework, not both TensorFlow and PyTorch.

- Run notebook visuals with Matplotlib instead of desktop-only `cv2.imshow`.
- Keep training/data download credentials out of committed notebooks.
- Honor dataset attribution/access terms.
- Cache features and load model checkpoints for the demo.
- Download/package required public data and model assets before the final presentation so inference can run without a live API.
- Export package versions and seeds. Note nondeterministic operations where applicable.

19. Three-person work allocation

Assign the following responsibility areas among Riddhi, Om, and Sudhan according to availability; the allocation is proposed, not an agreed personal assignment.

Owner slot	Primary responsibility	Deliverables	Shared dependency
Member A	Classical processing and texture	Notebooks 01â€“03; filters, colors, edges, masks, features, pyramids	ROI conventions, selected demo/development images
Member B	Data audit and condition classification	Notebooks 00 and 05; manifests, grouping, labels, model, evaluation	Feature interface and final label/task decision
Member C	Measurement, stereo, face, integration	Notebooks 04, 06â€“08; count/width, synthetic geometry, Haar, result panel	Stable candidate-mask and feature outputs

Member C has the broadest integration workload. Member A supplies reusable mask/skeleton helpers and helps the count/width notebook. Member B owns result-schema/model-loading checks and helps integrate classification into the final demo.

All three jointly review the label/split audit on Day 1 and the final evidence on Day 6. Each must be able to explain the complete pipeline, not only their own notebook.

20. One-week execution plan

If work starts on 13 October 2026, seven consecutive days end on 19 October 2026. These are planning dates, not a confirmed submission deadline.

Day	Main goals	Decision/checkpoint
1	Inspect/download one usable primary linked dataset; choose labels/task; check licenses, groups, hardware; agree interfaces	Freeze primary source, count independent groups, identify measurement/stereo/face-data gaps
2	Basic operations, colors, filters, edges; extract initial features; generate simple count/width/stereo fixtures	Shared loaders/ROI conventions work; first classical classifier trains
3	Threshold comparisons, texture, pyramids; first count/width estimates; stereo matching on synthetic scene	Every Lab 1â€“8 algorithm has at least an initial runnable output
4	Tune baseline on validation; complete measurement validity rules and Haar/no-face path; optional frozen MobileNetV2 extraction	All P0 module interfaces work; drop expensive P1 work if it threatens integration
5	Lock configurations; run controlled comparison and final evaluation; integrate analyzer and exports	No further test-driven tuning; selected model and figures frozen
6	Run notebooks from clean kernels; correct paths/plots/statuses; assemble gallery and faculty

explanations | One reliable demo, complete lab traceability, no unsupported measurement captions |
| 7 | Final verification, packaging, rehearsal, and buffer | Deliver notebooks, model/configuration, evidence, README, and scope/capability table |

20.1 If only three days remain

- Day 1: primary-source audit, basic/classical labs, initial classifier and deterministic verification fixtures.
- Day 2: count/width prototype, stereo/Haar modules, texture/pyramids, evaluation and integration.
- Day 3: clean execution, figures/exports, failure analysis, rehearsal.

Cut P1/P2 first: transfer fine-tuning, Grad-CAM, VGG19, dashboards, tracking, and frontend. Preserve the classical classifier, every lab-method implementation, and explicit data limitations. No schedule makes unavailable real calibration/labels appear.

20.2 Explicit stop rules

- Do not spend more than the initial data-audit day searching many new datasets; stay within the linked sources.
- Do not spend the week building a segmentation network from scratch.
- Stop transfer-learning work if downloads/runtime prevent a timely working baseline.
- Stop presenting biological count/physical width when the image does not support the prerequisites.
- Stop a real stereo branch at the input-validation gate if a pair/calibration is absent; run the synthetic verification branch.
- Add a frontend only after the final notebook application runs reliably.

21. Acceptance checklist

These items distinguish implementation completion from claims about real scalp measurements.

21.1 Data and reproducibility

- [ ] Selected dataset, actual version, license, label map, and independent-group counts are recorded.
- [ ] Raw inputs are preserved and derived crops keep parent identities.
- [ ] Train/validation/test or exploratory group-CV protocol is documented.
- [ ] Known derivatives/crops do not cross splits.
- [ ] Missing patient identity and other unresolved provenance limits are documented.
- [ ] Model, feature schema, class order, seed, configuration, and environment versions are saved.

21.2 Lab evidence

- [ ] Lab 1: load/display/resize/grayscale/pixel examples.
- [ ] Lab 2: mean/Gaussian/Laplacian-sharpening comparison.
- [ ] Lab 3: Sobel/Prewitt/Canny comparison.
- [ ] Lab 4: grayscale/HSV/LAB conversion and channel analysis.
- [ ] Lab 5: global/adaptive threshold comparison with overlays.
- [ ] Lab 6: contrast/energy/entropy numerical analysis.
- [ ] Lab 7: Gaussian/Laplacian levels and reconstruction check.
- [ ] Lab 8: real or clearly labeled synthetic pair, disparity/depth verification, and real-scalp capability status.
- [ ] Lab 9: Haar code and outputs; positive-case availability and manual/no-face behavior stated.
- [ ] Lab 10: integrated acquisition-to-visualization application.

Synthetic-only Lab 8 and a data-limited positive Haar demonstration must be disclosed to the faculty. They are not marked as real-data achievements.

21.3 Functional and scientific evidence

- [ ] A condition classifier is trained and appropriately evaluated against a baseline.
- [ ] At least one controlled comparison tests texture-feature contribution.
- [ ] Count and width algorithms are verified on known synthetic fixtures.
- [ ] Real measurement displays say exactly what is counted/measured.
- [ ] Physical width/density remains null without documented calibration and suitable data.
- [ ] Real scalp 3D remains unavailable without a valid pair/calibration.
- [ ] Point-cloud and synthetic figures have correct provenance and units.
- [ ] Empty masks, invalid ROIs, unreadable files, absent faces, and missing stereo do not crash the integrated application.

- [ ] At least three representative failure/limitation cases are explained.
- [ ] All notebooks run from clean kernels; the demo loads saved models.
- [ ] Final outputs contain no invented clinical accuracy, health score, treatment benefit, or calibrated measurement.

22. Faculty-facing explanation

22.1 Short project explanation

> We are building ScalpTrack, an explainable notebook-based system for scalp-condition classification and experimental hair analysis. We use every core lab technique: filters and color spaces prepare the images; edges and thresholds expose candidate hair/flake regions; texture features support a classical classifier; pyramids demonstrate analysis at different scales; Haar detection supports eligible full-head images; and a stereo module computes disparity and calibrated depth when valid pairs exist. We compare the methods, validate the measurement code on known fixtures, and report data limitations explicitly. Our public datasets support condition analysis, while true physical hair measurements need scale, labels, and appropriate stereo data.

22.2 What makes the project substantial

- A syllabus-wide technique comparison tied to one image-analysis problem.
- A reproducible classifier with grouped splits and a baseline.
- Count/width prototypes whose overlays expose their assumptions.
- Calibrated stereo computation with measurable synthetic ground truth.
- A complete result schema and integrated notebook application.
- Clear evidence of where each method succeeds and fails.

The strongest presentation is the quality of the experiments and evidence. Adding unsupported â€œ3D densityâ€ or â€œhealthâ€ numbers would weaken that presentation.

22.3 Questions the team should be able to answer

- Why does counting connected components not always count hairs?
- Why is apparent width in pixels different from diameter in micrometres?
- Why do histogram entropy and GLCM entropy differ?
- Why must augmented images and related crops stay in the same split?
- Why can thresholds mistake glare for flakes?
- Why does Haar fail on scalp close-ups, and what does it actually detect?
- What converts stereo disparity to metric depth?
- What does the synthetic stereo experiment prove, and what does it leave unproven?
- Which class labels were actually present in the downloaded dataset?
- Which outputs are validated, exploratory, synthetic, or unavailable?

23. Deferred extensions

23.1 Basic frontend

After P0 acceptance, a minimal local Streamlit-style wrapper can reuse the existing `analyze_image` function.

First-page scope: select/upload an image, choose ROI, run analysis, inspect overlays/scores/units/status, and export results. No new training or medical recommendations are introduced.

The frontend is not required for the first submission. Hosting, authentication, cloud deployment, and persistent user profiles are not part of this plan.

23.2 Progress tracking

Requires same-person repeated images with comparable scalp region, camera geometry, lighting, scale, and capture conditions. Public unrelated images cannot show an individual's improvement.

A later extension can register comparable images and show changes in descriptive coverage/width measures. Treatment causality and clinical improvement would require a separate validation study.

23.3 Real physical measurements and 3D

Acquire or locate explicitly approved data with labeled hairs/roots, high-resolution shaft images, physical scales, and calibrated stereo/3D reference measurements. Then evaluate count MAE,

physical-width error, density error, and depth error on real data.

This is the necessary upgrade path for the requested real measurements; it is not a task solved by changing the frontend.

23.4 General skin analysis

Reuse loading, color, texture, and classification infrastructure, but choose an appropriate labeled skin dataset and a separate task/evaluation plan. Scalp-condition labels cannot be reused as ground truth for arbitrary skin diseases.

24. References and evidence boundaries

24.1 Team-supplied context

- The lab list and email discussion pasted into this conversation.
- The attached `image(8).png`, containing the ScalpTrack concept.
- The linked Kaggle/Roboflow sources listed in Section 4.

Inaccessible notebook cells/shared discussion content have not been reconstructed or treated as reviewed implementation evidence.

24.2 Primary technical references

Use these to check API behavior and cite methods in the notebooks:

1. [OpenCV Image Filtering](https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html) â€” filtering and signed derivative operations.
2. [OpenCV Color Conversions](https://docs.opencv.org/4.x/de/d25/imgproc_color_conversions.html) â€” channel conversion and numerical encodings.
3. [OpenCV Image Thresholding](https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html) â€” fixed, Otsu, and adaptive methods.
4. [scikit-image GLCM and texture features](https://scikit-image.org/docs/stable/api/skimage.feature.html#skimage.feature.graycomatrix) â€” GLCM definitions, energy/ASM distinction, and feature extraction.
5. [OpenCV Image Pyramids](https://docs.opencv.org/4.x/d4/d1f/tutorial_pyramids.html) â€” downsampling/expansion and pyramid construction.
6. [OpenCV Camera Calibration and 3D Reconstruction](https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html) â€” pinhole geometry, rectification, Q, and disparity scaling.
7. [OpenCV Cascade Classifier](https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html) â€” Haar face detection and `detectMultiScale`.
8. [TensorFlow Transfer Learning and Fine-Tuning](https://www.tensorflow.org/tutorials/images/transfer_learning) â€” frozen/fine-tuned MobileNetV2 workflow.
9. [OpenCV Histogram Calculation](https://docs.opencv.org/4.x/d8/dbc/tutorial_histogram_calculation.html) â€” intensity/channel histogram analysis.

The dataset labels remain publisher-provided labels. No clinical adjudication or medical-performance verification was performed during this planning task.

25. Current capability commitment

Requested or discussed capability	Commitment for the one-week implementation
Scalp-condition classification	Primary functional deliverable, subject to a usable labeled linked export
Hair count	Implement clearly defined candidate-count/crossing analysis; real biological count remains conditional on suitable data
Hair thickness	Implement apparent width in pixels; physical width requires documented scale
3D measurements	Implement disparity/calibrated-depth subsystem and synthetic verification; real scalp 3D requires verified pair/calibration
Visible scalp/hair ratio	Supporting exploratory threshold-based descriptor, not true density
Dandruff percentage	Flake-candidate area only where appropriate; not a validated diagnosis/severity percentage
Scalp health/density index	Deferred until a defined, validated target and evidence exist
Haircare routine auditor	Deferred; no treatment recommendations or effectiveness claims

```
| Longitudinal monitoring | Deferred until comparable same-person longitudinal data exist |
| General skin analysis | Later phase, with a separate dataset/task |
| Frontend | Optional after notebooks pass the acceptance checks |

The implementation should finish with a clear capability table showing actual achievements. That
table is updated from executed evidence; this specification is the plan for obtaining it.
```